1. Architecture, Permanent Memory, and Buffer Management

   Key Concepts: Relational Engine vs. Storage Engine.

   Disk Access: Seek time, rotational latency, block transfer.

   Buffer Management: Buffer Pool, frames, page replacement (LRU), pinning and unpinning.

2. Primary Storage Organizations

   File Types: Heap, Sequential, Static and Dynamic Hashing (Virtual, Extensible, Linear).

   Trees: B-Trees and B+ Trees (Index Sequential).

   Cost Analysis: Estimating I/O costs (C_s, N_{pag}) for exact match, range search, and updates.

3. Secondary Organizations & Indexes

   Index Types: Clustered vs. Unclustered.

   Specialized Indexes: Inverted Indexes (for non-key attributes), Bitmap Indexes (for low cardinality), Multi-attribute Indexes.

4. Query Processing & Physical Operators

   Execution: Iterator Model (`open`, `next`, `close`).

   Operators: Filters, Joins (Nested Loop, Merge Join), Grouping, and External Merge Sort.

   Metrics: Cost (C) and Result Size (E_{rec}) estimation using Selectivity Factor (sf).

5. Query Optimization

   Phases: Analysis, Transformation, Physical Plan Generation.

   Equivalences: Pushing down selections and projections.

   Theory: Functional Dependencies, Armstrong's Axioms, Attribute Set Closure (X^+).

6. Transaction Management & Recovery

   Concepts: System vs. Media Failures, Write-Ahead Logging (WAL).

   Operations: Undo (Before Image) and Redo (After Image).

   Recovery: Checkpointing, Restart Algorithm (Backward Rollback, Forward Rollforward).

7. Concurrency Control

   Theory: Histories, Conflicting Operations, Conflict Serializability (Serialization Graph).

   Protocols: Two-Phase Locking (2PL), Strict 2PL.

   Advanced: Deadlock prevention (Wait-Die, Wound-Wait), Snapshot Isolation.

8. Physical Database Design and Tuning

   Methodology: Index selection process, subsumed index elimination.

   Tuning: Query rewriting (e.g., removing useless `GROUP BY`).

   Formulas: Detailed Selectivity Factor calculations for ranges and boolean conditions.

9. Decision Support Systems (DSS)

   Paradigms: OLTP vs. OLAP.

   Modeling: Multidimensional Conceptual Model (Facts, Dimensions, Measures).

   Schemas: Star and Snowflake.

SQL Extensions: `ROLLUP`, `CUBE`.

10. Column-Oriented Databases

    Architecture: Vectorization, CPU cache exploitation, Compression (RLE).

    Execution: Early vs. Late Materialization.

    Algebra: Column Algebra (MAL) operating on Binary Association Tables (BATs).

11. NoSQL & Distributed Systems

    Architectures: Parallel (Shared-Memory/Disk) vs. Distributed (Shared-Nothing).

    Distribution: Sharding and Replication (Master-Slave, Peer-to-Peer).

    Theory: CAP Theorem, Quorums.

    Models: Key-Value, Document, Column-Family, Graph, MapReduce.

Practice & Review

1. Exam Questions
   - **Column Databases:** Pros and cons, when should we use them?
   - **Concurrency:** Locks and granularity, why do we "need" intention locks?
   - **Concurrency:** Given a history H, determine if it is c-serializable and if it could be produced by a strict 2PL protocol.
   - **NoSQL:** Pros and cons, when should we use it?
   - **Distributed:** Difference between parallel systems and distributed systems.
   - **Data Warehousing:** Give conceptual and logical data mart designs. Specify if measures are additive, semi-additive, or non-additive. Give the SQL queries to produce the data of the reports.
2. Exercises
   - Calculation of operator costs and result sizes (E_{rec}) for given logical plans.
   - Functional Dependency closure computations.
   - ARIES Restart algorithm tracing given a specific log sequence.

01. Architecture

This chapter introduces the fundamental architecture of a Relational Database Management System (DBMS) and delves into the lowest levels of its storage engine: how data is permanently stored on disk and how it is temporarily cached in main memory for processing.

1. DBMS Architecture

A relational DBMS is traditionally divided into two main macro-components: the **Relational Engine** (or Query Processor) and the **Storage Engine**.

1.1 The Storage Engine

The Storage Engine handles the physical storage, retrieval, and management of data. It ensures data consistency, concurrency, and durability. It is composed of several layered sub-modules:

- **Permanent Memory Manager:** Manages the allocation and deallocation of space (pages) on the physical disk.
- **Buffer Manager:** Manages the transfer of pages between the permanent memory (disk) and the temporary memory (RAM).
- **Storage Structures Manager:** Implements the physical organization of records within files and pages.
- **Access Methods Manager:** Provides the algorithms (e.g., B+ Trees, Hash indexes, Sequential scans) to efficiently search and interact with the stored data.
- **Transaction & Concurrency Managers:** Guarantee ACID properties, handling locks, logging, and crash recovery.

---

2. Permanent Memory Manager (PMM)

The primary bottleneck in database operations is Disk I/O. The Permanent Memory Manager masks the hardware details of the physical disk, providing a logical abstraction to the upper layers.

> **Definition (PMM Abstraction):**
>
> The PMM provides a view of the permanent memory as a set of *databases*, where each database is a collection of *files*, and each file is an ordered sequence of fixed-size *pages* (or blocks).

2.1 Disk Access Costs

Data transfers between disk and RAM occur in units of **pages**. The performance of a DBMS is predominantly evaluated by the number of page transfers (I/O operations).

The physical cost of reading a page from a magnetic disk is defined as:

$$C = t_s + t_r + t_b$$

Where:

- t_s (Seek Time): Time to move the disk arm to the correct track.
- t_r (Rotational Latency): Time waiting for the correct sector to rotate under the head.
- t_b (Block Transfer Time): Time to read the data.

If we need to read k contiguous pages, the cost becomes:

$$C = t_s + t_r + k \cdot t_b$$

Because t_s and t_r are orders of magnitude larger than t_b, sequential reads are vastly more efficient than random access reads.

---

3. Buffer Management

The **Buffer Manager** is responsible for keeping the most frequently accessed pages in main memory (RAM) to minimize the expensive disk I/O operations.

3.1 The Buffer Pool

The memory allocated to the DBMS is organized into a **Buffer Pool**, which is an array of memory slots called **frames**. Each frame can hold exactly one disk page.

To track the status of each frame, the Buffer Manager maintains a directory (Resident Pages Table) containing metadata for each loaded page:

- `pinCount`: An integer indicating how many active transactions/processes are currently using the page.
- `dirty` **bit**: A boolean flag set to `true` if the page has been modified in RAM but not yet written back to disk.

3.2 Page Request Lifecycle

When the system requests a page P, the Buffer Manager executes `GetAndPinPage(P)`:

1. It checks if P is already in a frame in the Buffer Pool.
2. **If a hit occurs:** It simply increments the `pinCount` of that frame and returns the memory address.
3. **If a miss occurs:**
   - It searches for a free frame.
   - If no free frames exist, it invokes a **Replacement Policy** (typically **LRU** - Least Recently Used) to select a victim frame.
   - **Condition for replacement:** A page can *only* be evicted if its `pinCount == 0`.
   - If the victim page has `dirty == true`, the Buffer Manager must synchronously execute `FlushPage(P)` to write the modifications back to the disk before overwriting the frame.
   - Finally, it loads the requested page P into the chosen frame, sets `pinCount = 1`, `dirty = false`, and updates the directory.

3.3 Releasing Pages

When a process finishes using a page, it calls `UnpinPage(P)`, which decrements the `pinCount`. If a transaction modified the page, it must call `SetDirty(P)` to ensure the Buffer Manager knows the data is no longer synchronized with the disk.

> **Important Rule (Pinning):**
>
> A page with `pinCount > 0` is said to be *pinned*. Pinned pages cannot be evicted. The careful management of pinning and unpinning is critical; failing to unpin a page leads to buffer starvation.

02. Primary Storage Organizations

This chapter covers the fundamental primary organizations used by the Storage Structures Manager to store records in permanent memory. A primary organization dictates the physical placement of new records on disk.

---

1. Records and Pages

Records are mapped to fixed-size pages. Each record is uniquely identified by a **Record Identifier (RID)**, typically composed of `(PageID, SlotIndex)`. To avoid updating RIDs across the entire database when a record is moved within a page, pages use a **Slot Array**. The Slot Array contains the internal memory addresses of the records. Thus, if a record is updated and shifted inside the same page, only its Slot Array entry is updated, leaving the global RID unchanged.

1.1 Cost Parameters

The performance of data organizations is evaluated primarily by the number of I/O page transfers. * N_{pag}: Total number of pages allocated to the file. * N_{rec}: Total number of records. * D_{pag}: Size of a single page. * L_r: Length of a single record. * sf: Selectivity factor, defined as $\frac{k_2-k_1}{k_{max}-k_{min}}$ for range queries. * C_s: Base cost of a search operation.

---

2. Heap Organization

The most basic storage method. Records are appended strictly in their order of insertion.

- **Insertion:** Very efficient. The system retrieves the last page, inserts the record, and writes it back. Cost: 2 **page accesses** (1 read, 1 write).
- **Equality Search:** Requires a linear scan.
  - If the key exists: $C_s = \frac{N_{pag}}{2}$ (on average).
  - If the key does not exist: $C_s = N_{pag}$.
- **Range Search:** Unsorted data forces a full scan. Cost: N_{pag}.
- **Deletion/Update:** First, locate the record (C_s), then write the modified page to disk. Cost: $C_s + 1$.

---

3. Sequential Organization

Records are physically stored in sequential order based on a specific attribute (e.g., primary key).

- **Equality Search:** If pages are contiguously allocated, a binary search can be applied. Cost: $C_s = \log_2(N_{pag})$.
- **Range Search:** Highly efficient because records falling in the range are adjacent. Cost: $C_s = \log_2(N_{pag}) + \lceil sf \times N_{pag} \rceil - 1$.
- **Insertion:** This is the major drawback. Inserting a record in the middle of a full page causes a cascading shift of records across subsequent pages. Cost: Worst case $C_s + N_{pag} + 1$.
- **External Sorting:** To initially build or reorganize a sequential file, the DBMS uses External Merge Sort. The cost of sorting a file is approximately $2 \cdot N_{pag} \cdot$ (**Number of Merge Phases** $+ 1$).

---

4. Hashing Organizations

Records are placed into pages (buckets) using a hash function $H(k)$ applied to a specific key. This provides extremely fast point queries but destroys any sequential sorting.

4.1 Static Hashing

A fixed number of pages M is allocated initially. * **Equality Search:** Ideal cost is 1 **access**. * **Overflows:** Occur when collisions fill a page. Handled via *Open Overflow* (linear probing) or *Chained Overflow* (linked lists of overflow pages). Overflows severely degrade the $O(1)$ search cost. * **Reorganization:** When degradation is too high, a complete file reorganization is mandatory. Sorting hashes in memory before writing back reduces the reorganization cost to $4 \cdot N_{pag}$. * **Range Search:** Entirely unsupported (forces a full scan).

4.2 Dynamic Hashing

Addresses the static hashing limitations by allowing the hash space to grow gracefully, thus avoiding massive periodic file reorganizations. * **Virtual Hashing:** Doubles the primary space when an overflow occurs and dynamically adapts the hash function. It uses a bit vector to track allocated pages. It can suffer from a sparse hash table and low load factor (average 67%). * **Extensible Hashing:** Utilizes an auxiliary structure called a *Directory* (of size 2^p, where p is the prefix length of the hash used). When a page overflows, only that

specific page is split, and the directory may double to accommodate the new pointers. * **Linear Hashing:** The hash table grows linearly (one bucket at a time) rather than doubling. An overflow pointer P tracks which bucket is next to split, regardless of where the collision actually happened. A secondary hash function $H_1(k) = k \bmod 2M$ is used for the split buckets.

---

5. Tree Primary Organizations

To combine the search efficiency of sorted data with the dynamic insertion capabilities of hashing, DBMSs use balanced tree structures. Let m be the order of the tree and h its height.

5.1 B-Tree

Keys and data (or pointers to data) are stored in both internal nodes and leaves. * **Node properties:** Every node must be at least 50% full. * **Equality Search:** Follows pointers down to a leaf. Worst case cost is h **accesses**. * **Insertion:** Always happens at a leaf. * *Best case:* Leaf has space. Cost: h **reads** + 1 **write**. * *Worst case:* Leaf is full, causing a cascading split up to the root. Cost: h **reads** + $2h + 1$ **writes**. * **Deletion:** * *Best case:* h **reads** + 1 **write**. * *Worst case (underflow causing merges):* $2h - 1$ **reads** + $h + 1$ **writes**.

5.2 B+ Tree (Index Sequential)

A crucial variation where internal nodes only store routing keys. Actual data records are stored exclusively in the leaf nodes. * **Leaf Linking:** Leaves are connected via a doubly-linked list, forming a fully sequential sorted file at the bottom level. * **Equality Search:** Consistently costs h **accesses**. * **Range Search:** Outstanding performance. The search traverses the tree to find the lower bound (h steps) and then horizontally scans the linked leaves. Cost: $h + \lceil sf \times N_{leaf} \rceil$ **accesses**.

03. Secondary Organizations & Indexes

This chapter focuses on Secondary Access Organizations, commonly referred to as **Indexes**. While a primary organization determines the physical placement of records on disk, a secondary organization provides alternative access paths to the data based on different search keys.

An index is essentially a set of entries. An entry typically binds a search key value to the physical location of the corresponding record(s): `(Value, RID)`. Secondary indexes are predominantly implemented using B+ Trees or Hash tables.

---

1. Clustered vs. Unclustered Indexes

The most critical distinction in secondary indexes relies on how the index ordering relates to the physical data ordering.

1.1 Clustered Index (Index Sequential)

An index is **clustered** if the data records in the primary organization are physically sorted in the same order as the search key of the index. * **Constraint:** A table can have at most **one** clustered index because data can only be physically sorted in one way. * **Performance:** Highly efficient for range queries. Once the first matching record is found via the index, subsequent records are guaranteed to be physically adjacent on the disk pages. * **Cost (Range Search):** Search the index (h accesses) + read sequential data pages $\lceil sf \times N_{pag} \rceil$.

1.2 Unclustered (Non-Clustering) Index

An index is **unclustered** if the physical order of the data records does not match the logical order of the index key. * **Constraint:** A table can have multiple unclustered indexes. * **Performance:** Good for exact match (equality) queries returning very few records. Extremely expensive for range queries or low-selectivity queries, as every RID pointer might lead to a different random disk page access. * **Cost (Range Search):** Search the index (h accesses) + random data page accesses. In the worst case, every matching record requires a separate I/O operation: $h + \lceil sf \times N_{rec} \rceil$.

---

2. Inverted Indexes

Standard B+ tree indexes work well when the search key is a candidate key (unique values). However, if the index is built on a **non-key attribute** (e.g., `DepartmentID` in an `Employees` table), a single value will correspond to many RIDs.

Storing the pair `(Value, RID)` repeatedly for the same value wastes space. To optimize this, DBMSs use **Inverted Indexes**. * **Structure:** The index stores a single entry for each distinct value, pointing to a list of RIDs: `(Value, RID-List)`. * **RID-List management:** If the RID-list is small, it can be stored inside the index leaf node. If it is large, the leaf node stores a pointer to a separate chain of overflow pages containing the RID-list. * **Evaluation:** This drastically reduces the size of the index and speeds up the traversal. When executing a query, the system retrieves the entire RID-list for the requested value and then fetches the data pages.

---

3. Bitmap Indexes

For attributes with very **low cardinality** (a small number of distinct values, e.g., `Gender`, `Region`, `Status`), traditional B+ Trees are highly inefficient. **Bitmap Indexes** are the optimal solution for these cases.

- **Structure:** For an attribute with k distinct values, the index maintains k bit vectors (bitmaps). Each bitmap has a length equal to the number of records (N_{rec}) in the table.
- **Logic:** The i-th bit of the j-th bitmap is set to `1` if the i-th record has the j-th distinct value; otherwise, it is `0`.
- **Advantages:**
  - Highly compressed storage (especially with run-length encoding).
  - Extremely fast multi-condition query evaluation. Complex `WHERE` clauses combining `AND`, `OR`, and `NOT` can be resolved directly at the CPU level using bitwise logical operations on the bitmaps before ever touching the data pages.

---

4. Multi-Attribute Indexes

An index can be built on a search key composed of multiple attributes, e.g., `(A1, A2, A3)`. The order of attributes in the definition is crucial. The index sorts the entries lexicographically: first by `A1`, then by `A2` for resolving ties in `A1`, and so on.

- **Exact Match Query:** Extremely fast for conditions like `A1 = v1 AND A2 = v2 AND A3 = v3`.
- **Partial Match Queries:**
  - **Supported:** The index is very useful for queries involving a prefix of the search key, such as `A1 = v1` or `A1 = v1 AND A2 = v2`.
  - **Not Supported:** The index is practically useless for queries that do not specify the first attribute, such as `A2 = v2` or `A3 = v3`. Because the primary sorting is on `A1`, the values of `A2` are scattered across the entire index, forcing a full scan of all leaf nodes.

04. Query Processing & Physical Operators

This chapter describes how logical relational algebra expressions are translated into physical execution plans. A physical plan is a tree of physical operators provided by the Storage Engine.

---

1. The Iterator Model

To evaluate a query plan, DBMSs typically use a demand-driven pipeline model known as the Iterator Model (or Volcano model). Every physical operator implements a standard interface with the following methods:

- `open()`: Initializes the operator, allocates memory buffers, and recursively calls `open()` on its children to initialize the entire tree.
- `next()`: Returns the next record of the result. It drives the data flow from the leaves up to the root.
- `isDone()`: Checks if there are more records to produce.
- `close()`: Cleans up the state and releases allocated resources.

---

2. Cost and Result Size Estimation

The Query Optimizer needs to estimate the execution cost C (in terms of page I/O) and the expected number of resulting records E_{rec} for each physical operator.

The estimation relies heavily on the **Selectivity Factor (sf)**, which represents the probability that a record satisfies a given condition ψ.

---

3. Physical Operators for Selection (σ)

These operators filter records based on a predicate ψ.

3.1 Filter

Applies the condition on a generic input stream O (e.g., the output of another operator or a full table scan). * **Cost:** $C(O)$ (The cost to produce the input stream. The filtering itself happens in RAM). * **Result Size:** $E_{rec} = \lceil sf(\psi) \times E_{rec}(O) \rceil$

3.2 IndexFilter

Uses an unclustered secondary index I on relation R to evaluate ψ. * **Cost:** C_I (Cost to traverse the index and fetch the randomly scattered data pages). * **Result Size:** $E_{rec} = \lceil sf(\psi) \times N_{rec}(R) \rceil$

3.3 IndexSequentialFilter

Uses a clustered (index sequential) index I on relation R. Extremely efficient for range conditions because the data pages are physically contiguous. * **Cost:** $C_s + \lceil sf(\psi) \times N_{pag}(R) \rceil$ (where C_s is the initial index traversal). * **Result Size:** $E_{rec} = \lceil sf(\psi) \times N_{rec}(R) \rceil$

3.4 IndexOnlyFilter

Evaluates the query using *only* the index leaves, without ever accessing the actual data pages. Applicable when all attributes needed by the query (both for filtering and output) are present in the index key. * **Cost:** $C_s + \lceil sf(\psi) \times N_{leaf}(I) \rceil$

---

4. Physical Operators for Join ($\bowtie$)

Join operators take two inputs: an external operand O_E and an internal operand O_I, applying a join condition ψ_J.

4.1 NestedLoop

For each record in O_E, it scans the entire O_I to find matching records. * **Cost:** $C = C(O_E) + E_{rec}(O_E) \times C(O_I)$ * **Result Size:** $E_{rec} = \lceil sf(\psi_J) \times E_{rec}(O_E) \times E_{rec}(O_I) \rceil$

4.2 BlockNestedLoop (PageNestedLoop)

Optimizes I/O by loading a block (or page) of O_E into memory, then scanning O_I once for the entire block. Let B be the available buffer blocks. * **Cost:** $C = C(O_E) + \lceil \frac{N_{pag}(O_E)}{B} \rceil \times C(O_I)$

4.3 IndexNestedLoop

Uses an index on the join attribute of the inner relation O_I. Highly efficient if O_E is relatively small and O_I has a suitable index. * **Cost:** $C = C(O_E) + E_{rec}(O_E) \times (C_I + C_D)$ *(Where C_I is the index lookup cost and C_D is the data access cost for a single match).* * **Asymmetry Constraint:** The `IndexNestedLoop` is strictly asymmetric. The left-hand side (external operand) can be an arbitrarily complex tree of operators (e.g., a table scan or the result of another join), whereas the right-hand side (internal operand) must strictly be an `IndexFilter` based on the exact same join condition.

4.4 MergeJoin

Requires both O_E and O_I to be **sorted** on the join attributes. It scans both inputs simultaneously in a single pass (like the merge step in MergeSort). * **Cost:** $C = C(O_E) + C(O_I)$ *(Note: If the inputs are not already sorted, the cost of the `Sort` operator must be added).*

4.5 Bit-based Algorithms (Existence / Semi-Joins)

For specific operations checking the existence or absence of associations (e.g., finding all students who did not take any exams), the system can employ highly optimized bit-array algorithms instead of standard joins. * **Mechanism:** The DBMS allocates a single bit of memory for each entity of the outer relation (e.g., one bit per student, initialized to 0). * **Execution:** It then scans the inner relation (e.g., the exams table), setting the corresponding student's bit to 1 whenever an exam record is found. * **Result:** At the end of the scan, the system checks the bits remaining at 0 to output the missing information, filtering the results with an extremely minimal memory footprint.

---

5. Grouping (γ) and Duplicate Elimination (δ)

5.1 GroupBy & Distinct (Stream-based)

These operators require the input O to be **already sorted** (or at least grouped) on the grouping attributes. They operate sequentially in $O(1)$ memory by comparing the current record with the previous one. * **Cost:** $C = C(O)$ * **Result Size (Group By):** $E_{rec} = \min(\frac{E_{rec}(O)}{2}, \prod N_{key}(A_i))$

5.2 HashGroupBy & HashDistinct

If the input is not sorted, a hash table is built in the main memory buffer. If the hash table exceeds the available memory, partitions are written to disk (spilled), causing additional I/O overhead. * **Cost:** $C = C(O) + 2 \times N_{pag}(O)$ *(Assuming a single spill-to-disk phase).*

---

6. Sorting (τ)

The **Sort** operator physically orders the records. Since relations are generally too large to fit in RAM, the DBMS uses **External Merge Sort**. * **Cost:** $C = C(O) + 2 \times N_{pag}(O) \times$ (number of passes) *(The records are read into memory, sorted in runs, written to disk, and then merged in subsequent passes).*

6.1 Sorting with Aggregations (Redundancy)

When evaluating physical plans that require sorting on an attribute (e.g., `A`) followed by an aggregation, the optimizer must carefully determine if a subsequent requested sort on `(A, Aggregate)` is redundant. * **The Problem:** If a query requires the final output to be sorted by `R.A` and `COUNT(*)`, and the data stream was already sorted by `R.A` before the `GROUP BY`, the final sorting step on `COUNT(*)` is only redundant if `R.A` functionally determines the `COUNT(*)`. * **Re-sorting Requirement:** If `R.A` is not a strict key for the grouping operation (meaning the same value of `R.A` can be associated with different counts because other dimensions were removed from the output), the output will *not* be automatically sorted by the aggregate value. The DBMS must therefore inject an additional physical **Sort** operator after the grouping step to fulfill the query's ordering requirements.

05. Query Optimization

This chapter explores the Query Optimizer, the component responsible for translating a declarative SQL query into the most efficient physical execution plan possible. The optimization process is typically divided into three main phases: Query Analysis, Query Transformation, and Physical Plan Generation.

---

1. Query Processing Phases

1. **Query Analysis:** The DBMS parses the SQL query, verifies its syntactic and semantic correctness, and checks user authorizations against the system catalog.
2. **Query Transformation (Logical Optimization):** The query is translated into an internal relational algebra representation. The optimizer applies equivalence rules to rewrite the logical plan into a more efficient, semantically equivalent form. This phase is strictly independent of any physical data structures or indexes present in the database.
3. **Physical Plan Generation:** The optimizer maps the logical operators to specific physical operators. It is only in this phase that decisions regarding physical access paths are made (e.g., converting a logical filter into an `IndexFilter` or a `TableScan` based on available indexes). Furthermore, it selects specific algorithms based on properties like data ordering: for instance, choosing a `MergeJoin` over a `HashJoin` when multiple joins are cascaded, as `MergeJoin` exploits and preserves existing sorting for subsequent steps, whereas `HashJoin` does neither.

---

2. Physical Plan Generation and Search Space

When translating a multi-join logical query into a physical plan, the optimizer faces an exponentially large search space. For n joined relations, it must choose the permutation (order of evaluation), the associativity (tree shape), and the specific physical algorithm (e.g., Nested Loop, Merge Join, Hash Join) for each individual join, resulting in 3^n physical combinations just for the operators.

2.1 The System Catalog and Statistics

To evaluate and compare the costs of these millions of potential plans, the optimizer relies on the **System Catalog**, a meta-database containing statistical metadata about the tables. * It stores the number of pages, number of distinct keys, min/max values, and detailed **histograms** to precisely estimate Selectivity Factors.

* DBAs manually trigger the `UPDATE STATISTICS` command to realign this catalog when data distributions change over time.

2.2 Search Algorithms and Heuristics

To navigate the immense search space without taking too much time, optimizers employ different strategies: * **Complete Algorithms:** They use dynamic programming or branch-and-bound techniques, starting with single-relation access plans and incrementally expanding the cheapest ones. They backtrack if an expanded plan becomes more expensive than a previously explored branch, guaranteeing the discovery of the absolute fastest plan. * **Left-Deep Trees Heuristic:** To drastically prune the search space, the optimizer may restrict the tree shape to "Left-Deep Trees", a linear structure where the right-hand operand of every join must be a base table. This limits complexity and maximizes the continuous use of `IndexNestedLoop` operators. * **Greedy Search:** The optimizer picks the cheapest local option at every step and expands it downward without ever backtracking. * **Timeouts and Local Manipulations:** For extremely complex queries, the system rapidly generates a sub-optimal plan using heuristics, then applies random or localized structural manipulations to improve it until a strict timeout (e.g., 0.5 seconds) is reached, ensuring the user does not wait indefinitely for the optimization phase itself.

---

3. Relational Algebra Equivalences

The Query Transformation phase relies heavily on equivalence rules to optimize the logical plan. The most crucial heuristic is to **push down selections and projections** as close to the leaves of the tree as possible to reduce the size of intermediate results.

3.1 Selection Rules

- **Cascading of selections:**
$$\sigma_{\psi_X}(\sigma_{\psi_Y}(E)) \equiv \sigma_{\psi_X \wedge \psi_Y}(E)$$
- **Commutativity of selection and projection:**
$$\pi_Y(\sigma_{\psi_X}(E)) \equiv \sigma_{\psi_X}(\pi_Y(E)) \quad \text{if } X \subseteq Y$$
If X contains attributes not in Y, the rule becomes:
$$\pi_Y(\sigma_{\psi_X}(E)) \equiv \pi_Y(\sigma_{\psi_X}(\pi_{X \cup Y}(E)))$$
- **Commutativity of selection and join:** If the condition ψ_X only involves attributes of E_1:
$$\sigma_{\psi_X}(E_1 \bowtie E_2) \equiv \sigma_{\psi_X}(E_1) \bowtie E_2$$
If ψ_X involves attributes of E_1 and ψ_Y involves attributes of E_2:
$$\sigma_{\psi_X \wedge \psi_Y}(E_1 \bowtie E_2) \equiv \sigma_{\psi_X}(E_1) \bowtie \sigma_{\psi_Y}(E_2)$$

3.2 Pushing Selections through Group-By (HAVING to WHERE)

A specialized selection rule involves the `HAVING` clause (σ operating after a γ). * **Dimension Filters:** If a condition in the `HAVING` clause references *only* grouping dimensions (e.g., `HAVING Year > 2000`), the optimizer commutes the selection, pushing it below the `GROUP BY` into the `WHERE` clause. While human programmers naturally write dimension filters in the `WHERE` clause, this algebraic equivalence is crucial for optimizing machine-generated queries created by graphical Business Intelligence (BI) tools. * **Existential Quantification via Aggregate Functions:** Generally, conditions on aggregate functions cannot be pushed down. However, a major exception exists for existential checks using `MAX` or `MIN`. If a query filters groups with `HAVING MAX(A) > c` (and computes no other conflicting aggregate functions), it is logically checking if *at least one* record has `A > c`. The optimizer can safely push this filter down as a `WHERE A > c` condition *before* the `GROUP BY`, discarding irrelevant records early and drastically reducing the data to be grouped.

3.3 Subquery Unnesting and the "Count Bug"

Optimizers attempt to flatten nested subqueries (like `EXISTS`) into standard joins to avoid re-executing the inner query iteratively (nested loops) for every outer tuple. * Flattening an `EXISTS` subquery usually requires adding a `DISTINCT` clause to avoid duplicating rows from the outer query if multiple matches exist. * **The "Count Bug":** A critical edge case occurs when the subquery contains a trivial (implicit) `GROUP BY` with the condition `COUNT(*) = 0`, or equivalently, a `NOT EXISTS` clause. Unlike other aggregate functions, `COUNT(*)` applied to an empty set returns a row with the value zero rather than an empty result. * Transforming `COUNT(*) = 0` using a standard `INNER JOIN` is semantically incorrect because it completely eliminates the unmatched rows from the result. To correctly unnest this pattern, the optimizer must use a `LEFT OUTER JOIN` and subsequently filter for `NULL` values to track the entities with zero associations.

3.4 View Merging and Join Pushing

When querying a view, the optimizer tries to merge the view's logical plan with the outer query's plan to create a single, standard SQL plan where all joins are performed before grouping operations. * A structural challenge arises if the view contains a `GROUP BY` (γ) and the outer query performs a `JOIN` ($\bowtie$), resulting in an inverted plan where the join sits on top of the grouping operator. * **Join Pushing:** The optimizer can push the `JOIN` below the `GROUP BY` if and only if the join is "unary" (non-multiplicative). This rule strictly requires the join condition to equate a Foreign Key in the grouped data to the Primary Key of the joined table. Since the relation is on a primary key, each grouped row is multiplied by at most one, leaving the cardinality and the results of aggregate functions (like `SUM` or `COUNT`) uncorrupted. * **Dimension Expansion:** When the join is pushed, the new `GROUP BY` must expand its dimensions to include all attributes of the joined table.

3.5 Pre-Grouping (Pushing Group-By through Join)

While logical optimization usually pushes joins down (View Merging), the optimizer might evaluate the exact opposite transformation during physical plan generation if it determines that joining large tables is too expensive. * **The Concept:** Pre-grouping moves the `GROUP BY` operator *below* the `JOIN` to drastically reduce the number of records before the join is computed. * **Conditions:** Similar to View Merging, this is only valid if the table being extracted from the group-by is functionally determined by the grouping dimensions (i.e., the grouping attributes contain the foreign key linking to that table). Furthermore, all aggregate functions in the `SELECT` clause must operate exclusively on the table being pre-grouped, not on the table being pulled out of the grouping phase.

---

4. Functional Dependencies

During query optimization, the DBMS utilizes semantic constraints, specifically Functional Dependencies (FDs), to eliminate redundant operations (like useless `DISTINCT` or `GROUP BY` clauses) and simplify conditions.

> **Definition (Functional Dependency):** Given a relation schema R and subsets of attributes $X, Y \subseteq R$, a functional dependency $X \to Y$ (X determines Y) holds if, for any two tuples t_1, t_2 in R, $t_1[X] = t_2[X] \implies t_1[Y] = t_2[Y]$.

A special case is $\emptyset \to Y$, meaning the value of Y is the same for every tuple in the relation.

4.1 Evaluating Redundancy with FDs

Functional Dependencies are fundamental for proving structural equivalences and removing redundancies: * **DISTINCT and GROUP BY Elimination:** Transforming a query from a standard `SELECT` to an equivalent logical plan with `SELECT DISTINCT` requires the optimizer to check for potential row duplication (e.g., a teacher teaching multiple courses appearing multiple times in a join). By analyzing the keys and FDs, the optimizer can verify if the result is already guaranteed to be unique. If uniqueness is proven without the operator, the expensive duplicate elimination or grouping steps are completely removed from the execution plan. * **Preserving Group Cardinality in Join Pushing:** As seen in View Merging, pushing a join below

a `GROUP BY` requires expanding the grouping dimensions to include all attributes of the joined table. FDs mathematically prove why this expansion does not shrink the groups: because the join is performed on a Primary Key, all newly added attributes are functionally determined by the Foreign Key (which was already present in the original grouping dimensions). Therefore, the total number of distinct groups remains perfectly identical.

4.2 Logical Implication & Armstrong's Axioms

Given a set F of FDs, we can derive other dependencies that logically hold. $F \vdash X \to Y$ if it can be derived using Armstrong's axioms: 1. **Reflexivity:** If $Y \subseteq X$, then $X \to Y$. 2. **Augmentation:** If $X \to Y$ and $Z \subseteq T$, then $XZ \to YZ$. 3. **Transitivity:** If $X \to Y$ and $Y \to Z$, then $X \to Z$.

4.3 Closure of an Attribute Set (X^+)

Instead of repeatedly applying Armstrong's axioms, it is computationally simpler to compute the **closure** of an attribute set X with respect to F, denoted as X^+.

Theorem: $F \vdash X \to Y \iff Y \subseteq X^+$.

Algorithm to compute X^+ in Query Optimization: 1. Let $X^+ = X$. 2. Add to X^+ all attributes A_i such that the predicate $A_i = c$ is a conjunct of σ, where c is a constant. 3. Repeat until X^+ stops changing: * Add to X^+ all attributes A_j such that the predicate $A_j = A_k$ is a conjunct of σ and $A_k \in X^+$. * Add to X^+ all attributes of a table if X^+ contains a key for that table.

06. Transaction Management & Recovery

This chapter covers how the DBMS guarantees the Atomicity and Durability properties of transactions in the presence of concurrent executions and system failures.

---

1. ACID Properties and Failures

The Recovery Manager is responsible for handling database failures, focusing on two inherently asymmetric properties: * **Atomicity:** Pertains to failures *before* a transaction commits. It guarantees that if a transaction aborts or crashes mid-execution, none of its partial updates remain in the database (rollback). * **Durability:** Pertains to failures *after* a transaction commits. It guarantees that once the system confirms a commit, all updates survive any subsequent crashes.

1.1 Types of Failures

1. **System Failures:** Crashes caused by software bugs, OS panics, or power outages. The contents of the volatile main memory (Buffer Pool) are lost, but the permanent memory (disk) remains intact.
2. **Media Failures (Disasters):** Physical damage to the storage devices (e.g., disk head crash). These require restoring the database from a remote backup or an archive and applying the log.

---

2. The Log File and WAL

To recover from system failures, the DBMS maintains a **Log File** in permanent memory. The log is an append-only sequential file containing the history of all operations.

2.1 Log Records

Each update operation generates a log record containing: * **Transaction ID (T_{id})** * **Target Variable / Page ID** * **Before Image (BI):** The old value of the data, used for UNDO. * **After Image (AI):** The new value of the data, used for REDO. Format: `(W, Trid, Variable, Old value, New value)`.

2.2 Write-Ahead Logging (WAL)

To ensure that recovery is always possible, the Buffer Manager strictly adheres to the **Write-Ahead Logging (WAL)** protocol: > **WAL Rule:** Before a modified page P (with `dirty == true`) can be flushed from the volatile Buffer Pool to the permanent disk, all log records related to the updates on P must be forced to the permanent log file.

---

3. Undo and Redo Operations

Recovery algorithms rely on two fundamental operations applied to the log records: * **Undo:** Reverts the effects of an uncommitted (or aborted) transaction. It uses the Before Image (BI) and must be performed scanning the log **backwards**. * **Redo:** Re-applies the effects of a committed transaction that might not have been flushed to disk before the crash. It uses the After Image (AI) and must be performed scanning the log **forwards**.

3.1 Idempotence

A mathematical necessity for recovery operations is **idempotence**. Writing a Before Image (for Undo) or an After Image (for Redo) multiple times yields the exact same physical result as writing it once. This guarantees that if the DBMS experiences another system crash *during* the restart/recovery phase, the recovery process can simply be launched again from the beginning without corrupting the data.

---

4. Checkpointing

Scanning the entire log from the beginning during recovery is extremely inefficient. A **Checkpoint** operation periodically bounds the recovery time by guaranteeing that older updates are physically present on the disk.

4.1 Non-stop (Fuzzy) Checkpoint

To avoid freezing the entire database during a checkpoint, modern systems use an asynchronous "non-stop" algorithm: 1. The system writes a `begin checkpoint` record to the log, listing all transactions active at that exact moment. 2. A background thread begins sequentially flushing all currently dirty pages from the buffer to the disk. Meanwhile, regular transactions continue executing and modifying the buffer independently. 3. Once the thread finishes writing the specific batch of dirty pages, the system writes an `end checkpoint` record to the log. * **Guarantee:** If the restart algorithm finds an `end checkpoint`, it knows for absolute certainty that any write operation performed *before* the matching `begin checkpoint` is permanently on disk. If the system crashes before writing the `end checkpoint`, the pending checkpoint is simply ignored.

---

5. The Restart Algorithm

When the system reboots after a failure, the Recovery Manager executes a Restart procedure. The standard algorithm (e.g., ARIES-based logic) operates in passes.

5.1 Backward Pass (Rollback / Undo)

The system scans the log backwards starting from the end, until it finds the last valid `begin checkpoint` and all active transactions at the time of the crash are completely undone.

```
ckp = false;
toUndo = {};
toRedo = {};

for backward r in log until (ckp and empty(toUndo)) {
    if (r == (commit, T) and not ckp) {
        toRedo += {T};
    }
    elsif (r == (write, T, x, bi, ai) and not (T in toRedo)) {
        toUndo += {T};
        undo(x, bi);
    }
    elsif (r == (begin, T)) {
        toUndo -= {T};
    }
    elsif (r == (b-ckp, TList)) {
        ckp = true;
        toUndo += TList - toRedo;
    }
}
```

5.2 Forward Pass (RollForward / Redo)

Once the backward pass determines the exact sets of transactions to undo and redo, the system scans the log forwards starting from the last `b-ckp` record to reapply the updates of the committed transactions.

```
rollForward(toRedo):
for r in log starting from last begin-ckp until empty(toRedo) {
    if (r == (commit, T)) {
        toRedo -= {T};
    }
    elsif (r == (write, T, x, bi, ai) and (T in toRedo)) {
        redo(x, ai);
    }
}
```

5.3 Buffer-Centric Execution

It is crucial to note that the entire Undo and Redo execution during the Restart algorithm takes place strictly **inside the volatile Buffer Pool**. The recovery process restores the coherent state in the RAM, not forcing immediate writes to the disk. The physical disk itself only contains an outdated "photograph"; the true, up-to-date state of the database is continuously represented by the combination of the persistent log and the active buffer.

07. Concurrency Control

This chapter focuses on the Concurrency Manager, the module ensuring that the concurrent execution of transactions does not lead to inconsistencies. It introduces the theory of schedules (histories), serializability, and the locking protocols used to enforce it.

---

1. Transactions and Histories

A transaction T_i is a sequence of read ($r_i[x]$) and write ($w_i[x]$) operations on database elements, terminating with either a commit (c_i) or an abort (a_i). We ignore complex operations such as data creation or list insertions.

> **Definition (History / Schedule):** A history H on a set of transactions $T = \{T_1, T_2, \ldots, T_n\}$ is an ordered set of their operations that preserves the internal ordering of operations within each individual transaction.

1.1 Conflicting Operations

Two operations are in **conflict** if: 1. They belong to different transactions. 2. They access the same database item. 3. At least one of them is a write operation ($w_i[x]$).

1.2 Conflict Equivalence (c-equivalence)

Two histories H_1 and H_2 are **c-equivalent** if they are defined on the same set of transactions, have the same operations, and the relative order of all conflicting operations of normally terminated transactions is exactly the same.

---

2. Serializability

The primary goal of concurrency control is to ensure that a concurrent history is correct. A history is considered correct if its overall effect on the database is identical to a serial history.

2.1 Conflict Serializability (c-serializability)

A history H is **c-serializable** if it is c-equivalent to a serial history.

Theorem (Serialization Graph): To determine if a history H is c-serializable, we construct a Serialization Graph $SG(H)$: * **Nodes:** The committed transactions in H. * **Edges:** A directed edge $T_i \to T_j$ exists if an operation of T_i conflicts with an operation of T_j and strictly precedes it in H.

H is c-serializable **if and only if** its $SG(H)$ is **acyclic**. If the graph is acyclic, any topological sort of the nodes yields a valid equivalent serial history.

---

3. Locking Protocols

DBMSs use runtime protocols (schedulers) to restrict the generated histories to only serializable ones. The most common approach is based on locks.

3.1 Two-Phase Locking (2PL)

A transaction must acquire a read lock (shared) before reading an item and a write lock (exclusive) before writing. The 2PL protocol dictates that a transaction cannot request any new locks once it has released its first lock. This divides the transaction into two phases: * **Growing Phase:** The transaction acquires locks but cannot release any. * **Shrinking Phase:** The transaction releases locks but cannot acquire any new ones. **Result:** Any history produced by a 2PL scheduler is guaranteed to be c-serializable.

3.2 Strict 2PL

Standard 2PL can lead to *cascading aborts* if a transaction reads uncommitted data from a transaction that later aborts. **Strict 2PL** prevents this by enforcing that a transaction must hold all its exclusive (write) locks until it either commits or aborts.

---

4. Deadlock Management

Locking protocols can introduce **deadlocks** (e.g., T_1 waits for a lock held by T_2, and T_2 waits for a lock held by T_1). DBMSs handle deadlocks using timeouts, deadlock detection (cycle detection in a Waits-For graph), or prevention schemes based on transaction timestamps.

Two common prevention strategies are: * **Wait-Die:** Non-preemptive. If an older T_i requests a lock held by a younger T_j, T_i is allowed to wait. If a younger T_i requests a lock held by an older T_j, T_i "dies" (is aborted). * **Wound-Wait:** Preemptive. If an older T_i requests a lock held by a younger T_j, T_i "wounds" T_j (forces T_j to abort). If a younger T_i requests a lock held by an older T_j, T_i is allowed to wait.

---

5. Snapshot Isolation (SI)

Snapshot Isolation is a concurrency control strategy commonly used to increase read performance, offering an alternative to strict locking. * Each transaction operates on a consistent "snapshot" of the database taken when the transaction begins. * **Reads never block writes, and writes never block reads.** * To prevent lost updates, if two concurrent transactions attempt to write to the same item, the first one to commit succeeds, and the second one must abort. * **Write Skew:** SI prevents many concurrency anomalies but does not guarantee full conflict serializability, as it is vulnerable to *Write Skew* anomalies (where concurrent transactions read overlapping data but modify disjoint subsets based on stale snapshot data).

08. Physical Database Design and Tuning

This chapter covers the methodology used to select the optimal set of physical access structures (primarily indexes) for a given database workload, aiming to minimize the overall execution cost of queries and updates.

---

1. The Index Selection Process

Choosing the right indexes requires balancing the read performance benefits against the storage overhead and the update penalties (since every `INSERT`, `UPDATE`, or `DELETE` requires maintaining the indexes). The global view of the index selection process consists of five main steps:

1. **Identify critical queries:** Focus on the most frequent or resource-intensive queries in the workload.
2. **Create possible indexes:** Propose theoretical indexes that would perfectly tune each single critical query.
3. **Remove subsumed indexes:** Eliminate redundant indexes. An index I_1 on attribute A is generally subsumed by a multi-attribute index I_2 on (A, B), because I_2 can also be used to resolve searches on just A (as a prefix of the search key).
4. **Merge indexes where possible:** Instead of having one index on A and another on B, consider a single combined index on (A, B) or (B, A) to save space and update overhead, depending on the combined selectivity.
5. **Evaluate benefit/cost ratio:** For the remaining candidate indexes, calculate the mathematical cost of executing the workload with and without the index, factoring in the frequency of the queries versus the frequency of updates.

---

2. Review of Index Types for Design

When designing the physical layout, the DBA must choose between different index types based on the data cardinality and the query patterns.

- **Clustered Index:** Physically sorts the underlying data pages. You can only have one per table. Ideal for range queries (e.g., `Salary BETWEEN 30000 AND 50000`) or equality queries on non-unique attributes, because matching records will be physically contiguous.
- **Non-Clustered (Secondary) Index:** Provides pointers to physically scattered data. Good for exact match queries on highly selective attributes (e.g., `EmployeeID = 12345`).
- **Bitmap Index:** Extremely efficient for attributes with very low cardinality (e.g., `Country`, `Gender`, `Status`). It allows the DBMS to resolve complex boolean conditions (`AND`, `OR`, `NOT`) directly using bitwise operations before fetching the actual data records.

---

3. Database Tuning and Query Rewriting

Physical design is not just about adding indexes; it also involves tuning the queries themselves. The Query Optimizer handles many automatic transformations, but manual query rewriting is sometimes necessary to eliminate semantically useless operations.

- **Avoiding useless `GROUP BY` or `HAVING`:** A query like:

```sql
SELECT FkDepartment, MIN(Salary)
FROM Lecturers
GROUP BY FkDepartment
HAVING FkDepartment = 10;
```

Is highly inefficient because it groups the entire table first and then filters. It should be rewritten using selection pushing:

```sql
SELECT FkDepartment, MIN(Salary)
FROM Lecturers
WHERE FkDepartment = 10
GROUP BY FkDepartment;
```

(Note: If `FkDepartment` was not in the `SELECT` clause, the `GROUP BY` would become entirely useless and could be removed).

---

4. Selectivity Factor (sf) Formulas for Cost Evaluation

To correctly evaluate the benefit of an index in Step 5, you must estimate the Selectivity Factor (sf) of the query conditions. Assuming an attribute A with a known maximum ($max(A)$) and minimum ($min(A)$):

- **Equality ($A = c$):** $sf(\psi) = \frac{1}{N_{key}(A)}$
- **Range Inequality ($A < c$):** $sf(\psi) = \frac{c - min(A)}{max(A) - min(A)}$ (if A is numeric and has an index, otherwise typically estimated as 1/3).
- **Range BETWEEN (A BETWEEN c_1 AND c_2):** $sf(\psi) = \frac{c_2 - c_1}{max(A) - min(A)}$ (if A is numeric and has an index, otherwise 1/4).
- **Negation (NOT ψ):** $sf(\psi) = 1 - sf(\psi_1)$.
- **Conjunction (ψ_1 AND ψ_2):** $sf(\psi) = sf(\psi_1) \times sf(\psi_2)$ (assuming independent probabilities).
- **Disjunction (ψ_1 OR ψ_2):** $sf(\psi) = sf(\psi_1) + sf(\psi_2) - (sf(\psi_1) \times sf(\psi_2))$.

---

5. Estimating GROUP BY Cardinality

A critical part of physical design and query optimization is correctly estimating the number of groups produced by a `GROUP BY` operator to evaluate the memory and I/O footprint of operations like `HashGroupBy` or sorting.

5.1 Baseline Estimate vs. Range Filters

The theoretical maximum number of groups is the total number of distinct values of the grouping attribute in the table. However, if the query applies a range filter on that specific attribute, the estimate must be scaled down proportionally.

- **Example:** If attribute `A` has 10,000 distinct values globally, but a `WHERE` clause restricts `A` to a range that covers only 20% of its total domain, the estimated number of distinct groups generated by `GROUP BY A` will be 2,000 (i.e., $10,000 \times 0.20$).

5.2 Independence of Orthogonal Filters

When estimating the distinct values of a grouping attribute `A`, filters applied to *other* independent attributes (e.g., a highly selective condition on attribute `B`) should generally not be used to reduce the expected number of groups for `A`.

- **Reasoning:** Even if a filter on `B` eliminates 90% of the total records in the table, it randomly removes rows across all values of `A`. Unless there is a known strong statistical correlation between `A` and `B`, it is highly unlikely that restricting `B` will completely eliminate all instances of any specific value of `A`. Therefore, the estimated number of groups for `A` remains largely unaffected by orthogonal filters.

09. Decision Support System (DSS)

This chapter introduces Decision Support Systems and Data Warehousing, shifting the focus from traditional operational databases (OLTP) to systems designed for data analysis and business intelligence (OLAP).

---

1. OLTP vs. OLAP

- **OLTP (On-Line Transaction Processing):** Optimized for high-speed, concurrent transactions (inserts, updates, deletes). Focuses on current, detailed data and strict ACID properties.
- **OLAP (On-Line Analytical Processing):** Optimized for complex read-only queries (scans, aggregations). Operates on historical, consolidated data to support strategic decision-making.

---

2. The Multidimensional Conceptual Model

To design a Data Warehouse, we use a multidimensional model instead of a standard Entity-Relationship model. This model revolves around **Facts**, **Dimensions**, and **Measures**.

2.1 Facts

A **Fact** represents a business event or transaction that is worth analyzing (e.g., a sale, a hospital admission, a flight booking). * **Granularity:** The level of detail of a fact. It is best practice to choose the finest possible grain (e.g., an individual order line rather than a total daily order) to allow maximum flexibility in aggregation. If you focus on summarized orders initially, there is no way to do the analysis in reverse to move from measures about the orders to measures of individual lines.

2.2 Dimensions

Dimensions provide the context for the facts, addressing the classic "5W-1H" rules (Who, What, Where, When, Why, How). * Examples: `Customer`, `Product`, `Store`, `Time`. * **Hierarchies:** Dimensions often contain attributes organized in hierarchies (e.g., `Day` → `Month` → `Quarter` → `Year` or `City` → `Region` → `Country`). This allows users to drill down to details or roll up to summaries.

2.3 Measures

Measures are the numerical properties of a fact that can be aggregated and analyzed (e.g., `Quantity`, `Price`, `Revenue`, `Duration`).

Classification of Measures: 1. **Additive:** Can be meaningfully summed across *all* dimensions. (e.g., `Total Revenue`, `Quantity Sold`). 2. **Semi-additive:** Can be summed across *some* dimensions, but not all. Typically, they cannot be summed across the Time dimension. (e.g., `Inventory Level`, `Account Balance`). 3. **Non-additive:** Cannot be summed across *any* dimension. (e.g., `Percentages`, `Temperatures`, `Unit Price`).

---

3. Logical Design: Star and Snowflake Schemas

When translating the conceptual multidimensional model into a relational logical schema, two primary architectures are used:

3.1 Star Schema

The most common and efficient approach for OLAP queries. * Consists of a single, massive, highly-normalized **Fact Table** at the center. * Surrounded by completely **denormalized Dimension Tables**. * **Pros:** Simpler queries (fewer joins), faster read performance. * **Cons:** Redundancy in dimension tables (which is generally acceptable in Data Warehouses since data is rarely updated).

3.2 Snowflake Schema

A variation of the Star Schema where the dimension tables are **normalized**. * The central Fact Table remains the same, but dimension tables branch out into sub-dimensions (e.g., a `Store` table links to a separate `City` table). * **Pros:** Saves storage space by eliminating redundancy. * **Cons:** Queries require more complex and expensive joins, degrading performance.

---

4. Analytic SQL

Standard SQL is extended with analytic operators to easily compute subtotals and multidimensional aggregations without writing complex `UNION` queries.

- **`ROLLUP (A, B, C)`:** Computes the standard `GROUP BY (A, B, C)`, plus hierarchical subtotals: `(A, B)`, `(A)`, and the grand total `()`. Useful for time or geographical hierarchies.
- **`CUBE (A, B, C)`:** Computes the standard `GROUP BY`, plus *all possible combinations* of the attributes: `(A, B, C)`, `(A, B)`, `(A, C)`, `(B, C)`, `(A)`, `(B)`, `(C)`, and `()`. Used for cross-tabular analysis.

10. Column-Oriented Databases

This chapter explores Column-Oriented Databases (e.g., MonetDB, C-Store), an architectural paradigm shift designed primarily for OLAP and read-heavy workloads. Unlike traditional Row-Oriented DBMSs that store entire records contiguously, column stores write data to disk and memory one column at a time.

---

1. Architecture and Low-Level Efficiency

In a column-store, each attribute of a table is stored in a separate file or memory segment. This physical layout provides significant advantages for analytical queries that scan massive datasets but only access a few specific columns.

- **I/O Optimization:** The Storage Engine only loads the strictly necessary columns into memory, drastically reducing the disk I/O bottleneck.
- **CPU Cache & Vectorization:** Since data in a single column is entirely homogeneous (e.g., an array of contiguous 32-bit integers), the architecture highly exploits CPU caches and hardware prefetching. It allows for vectorized query execution, where low-level SIMD (Single Instruction, Multiple Data) instructions can process multiple values in a single CPU cycle.
- **Compression:** Homogeneous data compresses much better than heterogenous row data. Techniques like Run-Length Encoding (RLE) can represent consecutive identical values with a single `(value, count)` pair, allowing the execution engine to operate directly on compressed data.

---

2. Tuple Reconstruction

Because the attributes of a logical record are physically separated, the DBMS must perform **tuple reconstruction** to stitch the fields back together to serve the final query result.

2.1 Early Materialization

The system reconstructs the entire row immediately after scanning the necessary columns from disk, passing standard row-oriented tuples up the query execution tree. While simple, this negates many of the CPU and memory bandwidth advantages of column stores in the upper layers of the plan.

2.2 Late Materialization

Modern column stores keep the data in separate columns for as long as possible during query execution. Operators pass arrays of positions (Object IDs or oIDs) instead of actual data values. * **Advantage:** Intermediate results are highly compressed bitmaps or lists of integers. * **Execution:** Attributes are only fetched and stitched together at the very top of the query plan, right before returning the result to the user.

---

3. Column Algebra (MAL)

To natively support Late Materialization, systems like MonetDB use a specific column algebra where every operator works on Binary Association Tables (BATs). A BAT represents a column and consists of two fields: `bat[H, T]` (Head and Tail). The Head is typically the virtual Object ID (oID), and the Tail is the actual attribute value (or another oID).

3.1 Fundamental BAT Operators

- **`select(bat[H, T]_{AB}, bool * f(...))`**: Evaluates a predicate. It returns `bat[H, nil] = <[a, nil] | [a, b] \in AB \land f(b, ...)>`. It essentially extracts the oIDs of the records that satisfy the condition.
- **`reconstruct(bat[H, nil]_{AN}, bat[H, T]_{AB})`**: Fetches the actual values for a filtered set of oIDs. It returns `bat[H, T] = <[a, b] | [a, b] \in AB \land [a, nil] \in AN>`.
- **`join(bat[T1, T2]_{AB}, bat[T2, T3]_{CD}, bool * f(...))`**: Joins two columns on their matching tail/head values. It returns `bat[T1, T3] = <[a, d] | [a, b] \in AB \land [c, d] \in CD \land f(b, ...)>`.
- **`reverse(bat[H, T]_{AB})`**: Swaps the head and tail. It returns `bat[T, H] = <[b, a] | [a, b] \in AB>`.

- **`voidtail(bat[H, T]_{AB})`**: Discards the tail, useful for propagating only the oIDs. It returns `bat[H, nil] = <[a, nil] | [a, b] \in AB>`.
- **`group(bat[oID, T]_{AB})`**: Groups identical values. It returns `bat[oID, oID] = {[a, o] | o = id_{AB}(b) \land [a, b] \in AB}`.
- **`sum(bat[oID, int]_{AB})`**: Computes the sum aggregation. It returns a `bat[oID, int] = [nil, \sum \{i | [o, i] \in AB\}]`.

A typical query plan using MAL performs selections to generate lists of qualifying oIDs (`voidtail`), joins these oID lists, and ultimately calls `reconstruct` only for the attributes specified in the `SELECT` clause.

11. NoSQL & Distributed Systems

This chapter introduces the transition from centralized relational databases to distributed architectures and NoSQL systems, highlighting the trade-offs between consistency, availability, and scalability.

---

1. Parallel vs. Distributed Systems

A frequent exam topic is the distinction between parallel and distributed architectures: * **Parallel Systems:** Tightly coupled components operating within the same physical environment or high-speed local network. They often share resources like memory (Shared-Memory) or disk (Shared-Disk) to accelerate the execution of a single complex query. * **Distributed Systems:** Loosely coupled independent nodes connected via a network (Shared-Nothing architecture). Each node has its own memory and disk. The primary goals are horizontal scalability, high availability, and fault tolerance across different geographical locations.

---

2. Distribution Models

When moving data across a distributed cluster, systems employ two main techniques, often combined:

2.1 Sharding (Partitioning)

Data is divided into distinct subsets (shards), and each shard is assigned to a different node. * **Pros:** Horizontally scales read and write capacity. * **Cons:** Does not inherently provide high availability; if a node goes down, its shard is inaccessible.

2.2 Replication

Copies of the same data are stored on multiple nodes to ensure fault tolerance and increase read capacity. * **Master-Slave Replication:** One node (Master) handles all writes and synchronizes data to multiple Slaves. Slaves handle read requests. Good for read-heavy workloads but introduces a single point of failure for writes. * **Peer-to-Peer Replication:** All nodes are equal and can accept both reads and writes. Eliminates the single point of failure but drastically increases the complexity of resolving write conflicts.

---

3. Consistency and The CAP Theorem

In distributed systems, strict ACID properties are often relaxed to achieve scalability and performance.

3.1 The CAP Theorem

Formulated by Eric Brewer, it states that a distributed data store can simultaneously provide at most **two** of the following three guarantees: 1. **Consistency (C):** Every read receives the most recent write or an error. 2. **Availability (A):** Every request receives a non-error response, without the guarantee that it contains

the most recent write. 3. **Partition Tolerance (P):** The system continues to operate despite an arbitrary number of messages being dropped or delayed by the network.

Since network partitions (P) are inevitable in distributed systems, architects must essentially choose between **CP** (Consistency and Partition Tolerance, failing requests if the system cannot be synchronized) or **AP** (Availability and Partition Tolerance, returning potentially stale data).

3.2 Quorums

To manage consistency in Peer-to-Peer systems, Quorums are used. Let N be the number of replicas, W the write quorum, and R the read quorum. Strong consistency is guaranteed if:

$$R + W > N$$

This ensures that the sets of nodes written to and read from always overlap.

---

4. NoSQL Data Models

NoSQL databases deviate from the tabular relational model. They are typically chosen for their flexible schema, horizontal scalability, and performance on specific workloads.

1. **Key-Value Stores:** The simplest model. Data is a hash map of unique keys pointing to opaque values (blobs). Excellent for caching and fast lookups, but impossible to query by value.
2. **Document Databases:** Values are stored as structured documents (e.g., JSON, BSON). The database understands the internal structure, allowing queries on specific nested fields and the creation of secondary indexes.
3. **Column-Family Stores:** Data is stored in column families (rows that have many, varying columns). Highly optimized for massive write volumes and reading specific columns across many rows.
4. **Graph Databases:** Data is modeled as Nodes, Edges (relationships), and Properties. Unlike other NoSQL models, they prioritize relationship traversal (e.g., finding the shortest path or friends-of-friends) over horizontal scalability.

---

5. MapReduce

MapReduce is a programming model for processing and generating large data sets across a distributed cluster. It abstracts the complexity of parallelization, fault tolerance, and load balancing.

- **Map Phase:** The input data is split into independent chunks. The `Map` function processes these chunks in parallel, generating key-value pairs as intermediate output.
- **Shuffle & Sort:** The framework groups all intermediate values associated with the same intermediate key.
- **Reduce Phase:** The `Reduce` function aggregates the grouped values to produce the final output (e.g., summing totals, filtering).